

《云计算与大数据处理》

实验报告

专　　业：计算机科学与技术
班　　级：24 计科 3
学　　号：2410311308
姓　　名：龚治榕
实验（二）：数据湖与质量契约
实验日期：2026.10.27

《云计算与大数据处理》课程实验报告

实验名称	数据湖与质量契约					实验序号	02
姓　　名	龚治榕	专　业	计科	班　　级	24 计科 3	学　　号	2410311308
实验类型	综合型	实验日期	10.27	指导教师	刘锦行	成　　绩	

一、实验目的与环境

实验目的：

1.用十行以内的小表区分非法、重复和同 ID 冲突，并建立数量守恒关系。

2.运行脚手架生成 Silver 与 Gold 两层数据，用 SQL 核对总量和来源。

3.补全一项质量规则，用变式说明误拒绝和漏接收的代价。

4.比较 CSV、未压缩 Parquet 与 Zstd Parquet 的存储体积与查询耗时。

5.用清单哈希核验文件血缘，并说明清单与 ACID 事务的区别。

数据湖：集中保存不同类型、不同加工阶段数据的体系。本课程用本地三层目录学习组织与处理规则：原始层 Bronze 保留含坏行和重复的 JSONL，清洗层 Silver 只保留合法且 event_id 唯一的 Parquet,应用层 Gold 存放固定 SQL 聚合结果。这些层名表示加工职责，不表示不同物理机器。

质量契约：约定一条记录必须具备的字段、类型、范围和关系，既服务于生产数据的人，也服务于消费数据的人。本章使用的 DuckDB 是可在 Python 进程内运行的分析型 SQL 引擎，PyArrow 是内存表与 Parquet 之间的转换接口。

实验环境：硬件与系统基线

Windows 10.0.26200（64 位），CPU 运行，Python 3.12.10，沿用实验 01 的虚拟环境，未重新安装

关键依赖版本

duckdb 1.5.5、pyarrow 25.0.1、pandas 3.0.5、numpy 2.5.3，pip check 无冲突

本实验关键配置

课程根目录　E:\Desktop\2410311308_龚治榕_云计算实验\BDP，解释器 .venv\Scripts\python.exe，数据来源为实验 01 生成的 data/raw/events.jsonl

代码、截图与运行证据仓库：https://github.com/yiyun2026/bdp-coursework（点击可跳转）

二、实验内容与步骤

工作目录:2410311308_龚治榕_云计算实验\BDP(课程仓库根目录，个人修改在 submissions\lab02)

o 实验内容

实验 01 证明了文件经过 S3 API 往返后字节未变，但没有证明内容适合分析。本章在本人笔记本上分析合成服务记录：如果把重复请求算两次，统计会偏高；如果把负 Token 视作退款，统计可能偏低；如果无效时间混进分组，日统计会失真。本章把这些判断写成明确、可执行的规则，并让坏行进入隔离区而不是默默消失。

加工链路如下：

原始层 Bronze:保留收到的原始 JSONL,含坏行和重复 → 校验 + 隔离 + 明确去重 → 清

洗层 Silver：仅保留合法且 event_id 唯一的 Parquet → 固定 SQL 聚合 → 应用层 Gold：按日期与服务汇总的 Parquet

主键：唯一标识一个业务对象的字段，本课程用 event_id 标识事件。两行值相同但 ID 不同，可能是两次真实请求，不能仅因内容相似就删掉一条。幂等：相同操作重复执行不产生重复的业务效果。隔离区 quarantine：保存拒绝的数据和原因，让数据问题可追溯。数据血缘：记录某个产物来自哪份输入、经过什么规则与过程，本课程用清单哈希和契约版本表达最小血缘。

o 实验步骤

1. 前置检查与身份登记

1.1 设置编码并确认课程根目录

沿用实验 01 的 .venv，不重新安装环境、不重新生成原始文件。

```
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> cd E:\Desktop\241
0311308_龚治榕_云计算实验\BDP
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> $env:PYTHONUTF8 =
 '1'
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP>
```

设置 PYTHONUTF8，确认位于课程根目录

1.2 登记本人学号并检查前置数据

本章使用实验号 02。学号按字符串保存并保留前导零，登记后不允许覆盖为另一学号。

```
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> $studentId = Read
-Host '请输入本人学号'
请输入本人学号: 2410311308
PS E:\Desktop\2410311308_龚治榕_云计算实验\BDP> .\.venv\Scripts\p
ython.exe -m coursekit identity --student-id $studentId --lab 02
--stage start
{
  "STUDENT_ID": "2410311308",
  "EXPERIMENT_ID": "02",
  "IDENTITY_CODE": "6ac552b6b57a",
  "RUN_ID": "cddfee846a1549db9abe3f1190d26d54",
  "CAPTURED_AT": "2026-10-27T02:33:50.042926+00:00",
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311308",
    "experiment_id": "02",
    "run_id": "cddfee846a1549db9abe3f1190d26d54",
    "captured_at": "2026-10-27T02:33:50.042926+00:00",
    "stage": "start"
  },
  "stage": "start",
  "result": {
    "identity_note": "本人学号仅用于证据关联；每张评分截图必须带
学号，不能替代原创性验收。"
  },
  "evidence_json": "E:\\Desktop\\2410311308_龚治榕_云计算实验\\BD
P\\outputs\\lab02\\evidence\\cddfee846a1549db9abe3f1190d26d54-sta
rt.json",
  "evidence_html": "E:\\Desktop\\2410311308_龚治榕_云计算实验\\BD
P\\outputs\\lab02\\evidence\\cddfee846a1549db9abe3f1190d26d54-sta
rt.html"
}
```

学号 2410311308，实验号 02，IDENTITY_CODE = 6ac552b6b57a



Test-Path 输出 True，raw.json 记录生成规模与摘要



start 证据页，run_id cddfee846a1549db9abe3f1190d26d54

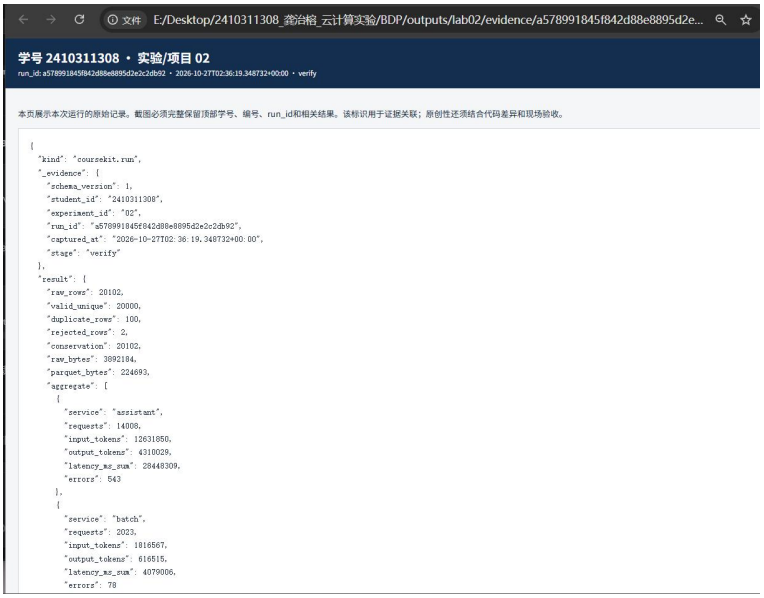
2. 运行主流水线

2.1 执行 lab02

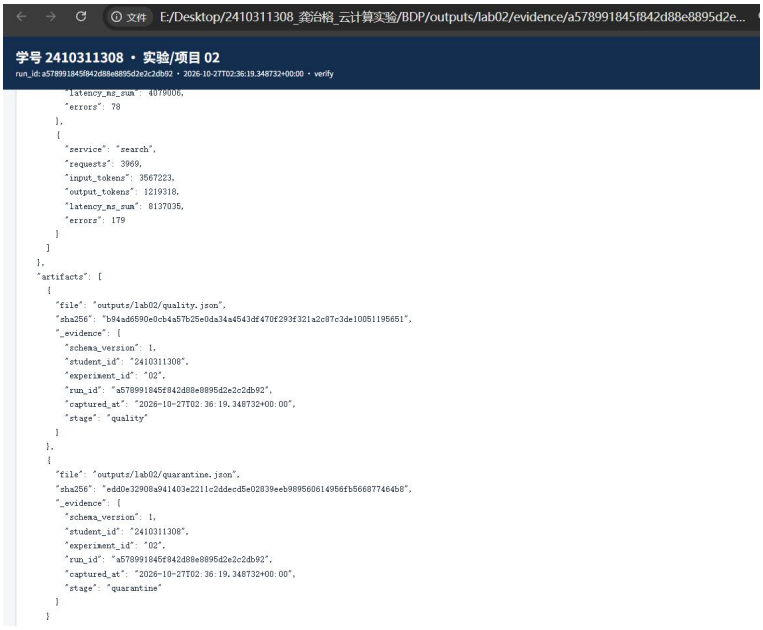
程序清洗数据、写出 Parquet、运行聚合并写出清单。



raw_rows=20102、valid_unique=20000、duplicate_rows=100、rejected_rows=2、conservation=20102



verify 证据页第一张，run_id a578991845f842d88e8895d2e2c2db92



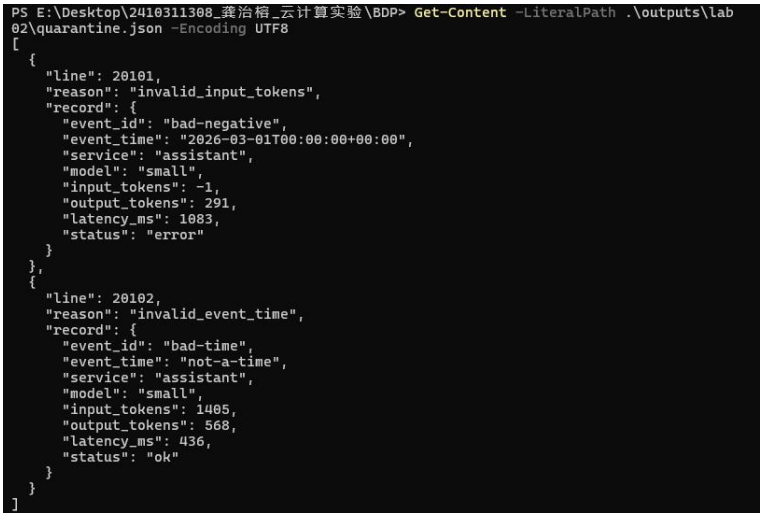
verify 证据页第二张，含三个服务聚合结果与产物哈希

2.2 查看质量、隔离与清单三个结果文件

quality.json 记录总量与守恒，quarantine.json 记录被拒行号、原因与原记录，silver.json 记录源文件、Silver、Gold 三个摘要与契约版本。



quality.json：20102 行原始数据，20000 合法独特、100 重复、2 隔离，守恒 20102



quarantine.json:第 20101 行 invalid_input_tokens,第 20102 行 invalid_event_time

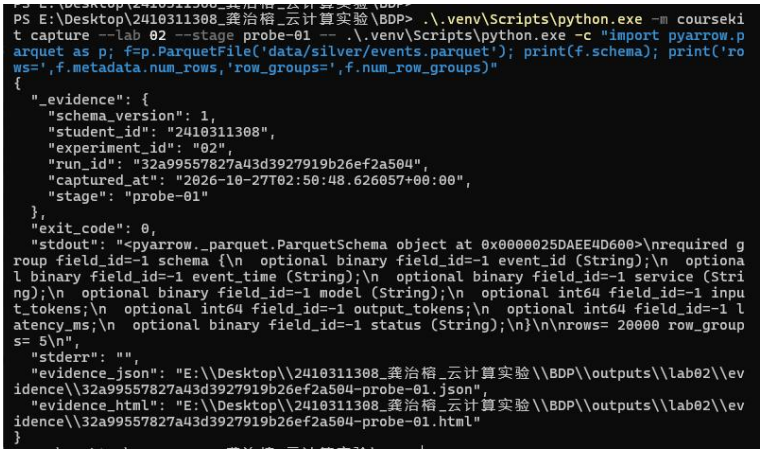


silver.json：契约版本 2，三份产物摘要齐全

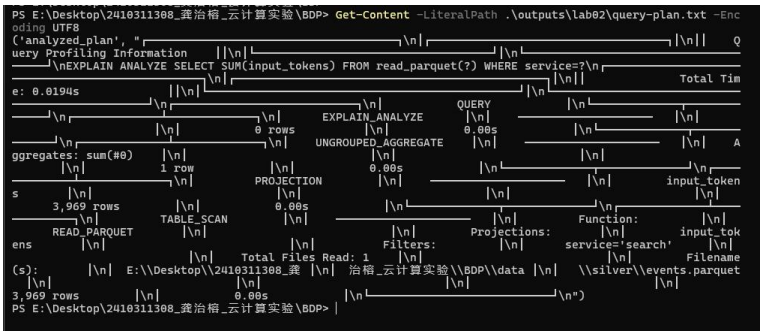
3. 观察 Parquet 结构与查询计划

3.1 读取 Parquet 元数据

默认应有 20000 行，按 4096 行组上限写入形成 5 个行组。



8 个字段的 schema，rows= 20000 row_groups= 5



probe-01 证据页，run_id 32a99557827a43d3927919b26ef2a504



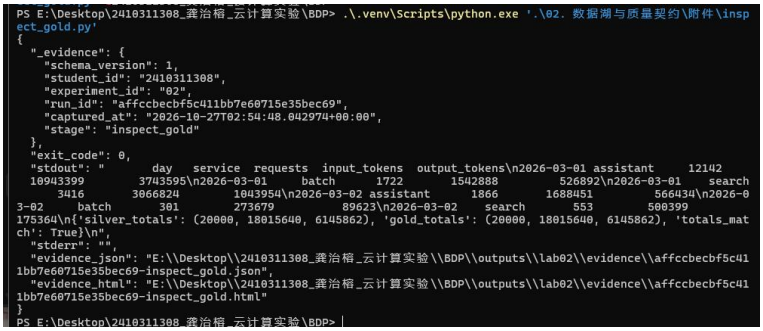
Parquet 结构探针证据页，含完整 stdout

3.2 阅读查询计划

查询计划来自 DuckDB 的 EXPLAIN ANALYZE，描述实际执行操作和测量信息。计划中 TABLE_SCAN 采用 READ_PARQUET，Projections 只有 input_tokens 一列，Filters 为 service='search'，这是列裁剪与谓词下推的直接证据。

4. 用 SQL 核对 Silver 与 Gold 总量

程序先对每个服务计算请求数、输入输出 Token 总量、延迟总和与错误数，再生成每日服务汇总。COUNT(*) 计入所有合法事件，包括 status='error' 的请求，因为请求失败仍可能消耗资源。



每日汇总表与总量核对，totals_match': True



inspect_gold 证据页，run_id affccbecbf5c411bb7e60715e35bec69

5. 个人改动：新的质量阈值

5.1 复制附件并保存基线

附件调用共用校验函数，只写个人结果，不改写供后续实验使用的 Silver。基线附加政策接受 input_tokens ≤ 2000，课程合成合法数据的输入量不超过 1800，因此这条附加规则不会拒绝已有合法事件。

```
PS E:\Desktop\2410311308_龚治棺_云计算实验\BDP> New-Item -ItemType Directory -Force .\submissions\lab02 | Out-Null
PS E:\Desktop\2410311308_龚治棺_云计算实验\BDP> Copy-Item -LiteralPath '.\02. 数据湖与质量契约\附件\student_quality_policy.py' -Destination .\submissions\lab02\student_quality_policy.py
PS E:\Desktop\2410311308_龚治棺_云计算实验\BDP> .\.venv\Scripts\python.exe .\submissions\lab02\student_quality_policy.py
{
  "_evidence": {
    "schema_version": 1,
    "student_id": "2410311308",
    "experiment_id": "02",
    "run_id": "4d92a580e80f43adb23a0303a7eb8303",
    "captured_at": "2026-10-27T02:56:34.162826+00:00",
    "stage": "student_quality_policy"
  },
  "exit_code": 0,
  "stdout": "{\n  \"max_input_tokens\": 2000,\n  \"raw_rows\": 20102,\n  \"valid_unique\": 20000,\n  \"duplicates\": 100,\n  \"rejected_by_reason\": {\n    \"invalid_input_tokens\": 1,\n    \"invalid_event_time\": 1\n  },\n  \"input_tokens_sum\": 18015640,\n  \"probe_errors\": {\n    \"negative\": \"invalid_input_tokens\",\n    \"boolean\": \"invalid_input_tokens\",\n    \"naive_time\": \"non_utc_time\"\n  },\n  \"_evidence\": {\n    \"schema_version\": 1,\n    \"student_id\": \"2410311308\",\n    \"experiment_id\": \"02\",\n    \"run_id\": \"4d92a580e80f43adb23a0303a7eb8303\",\n    \"captured_at\": \"2026-10-27T02:56:34.162826+00:00\",\n    \"stage\": \"student-policy-2000\"\n  }\n}\n",
  "stderr": "",
  "evidence_json": "E:\\Desktop\\2410311308_龚治棺_云计算实验\\BDP\\outputs\\lab02\\evidence\\4d92a580e80f43adb23a0303a7eb8303-student_quality_policy.json",
  "evidence_html": "E:\\Desktop\\2410311308_龚治棺_云计算实验\\BDP\\outputs\\lab02\\evidence\\4d92a580e80f43adb23a0303a7eb8303-student_quality_policy.html"
}
```

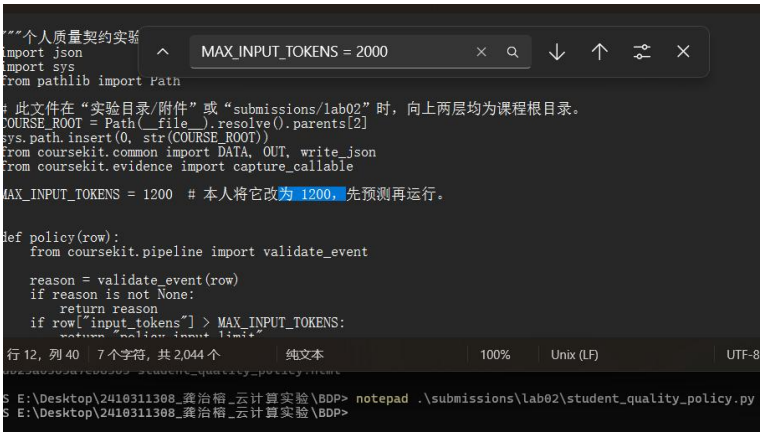
复制到 submissions/lab02 后原样运行，max_input_tokens 为 2000



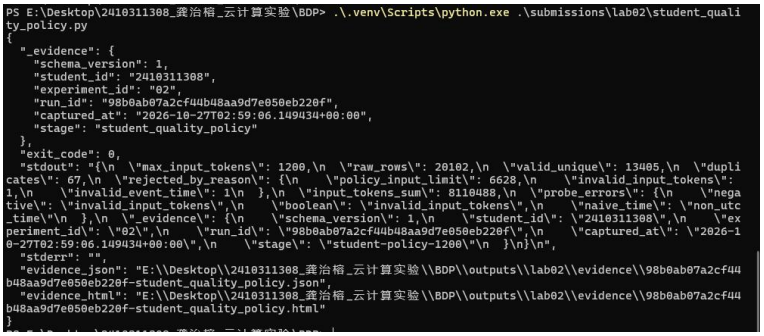
2000 阈值证据页，run_id 4d92a580e80f43adb23a0303a7eb8303

5.2 把阈值改为 1200

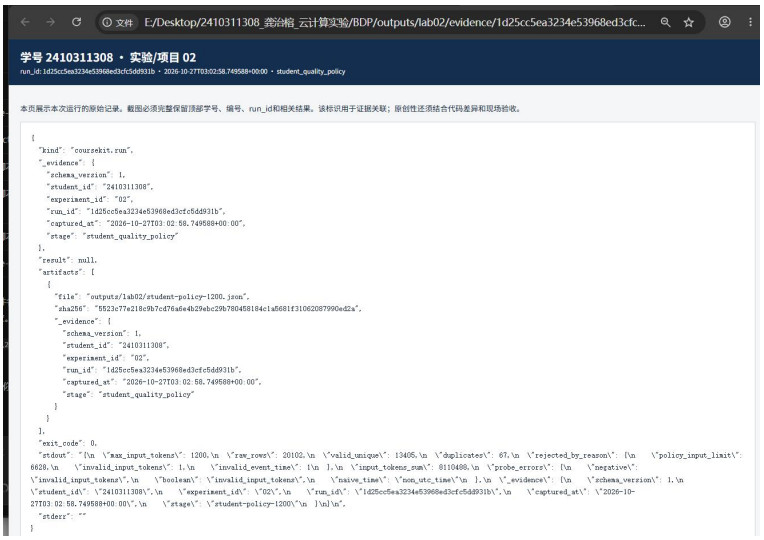
修改前先写出预测：合法独特数会减少，总输入 Token 会减少，重复数也可能减少——因为程序先校验再去重，被新阈值拒绝的原始重复行也进入拒绝计数。



将 MAX_INPUT_TOKENS 由 2000 改为 1200



1200 阈值运行结果，valid_unique 降至 13405



1200 阈值证据页，run_id 1d25cc5ea3234e53968ed3cfc5dd931b

5.3 生成并核对代码差异

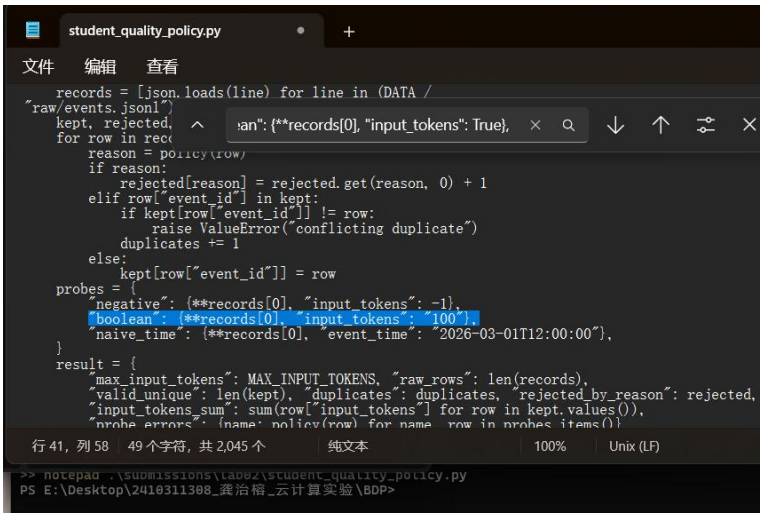
代码块先检查 Python 返回码，失败就停止；核验通过后，Git 返回 1 只表示发现预期源码差异。



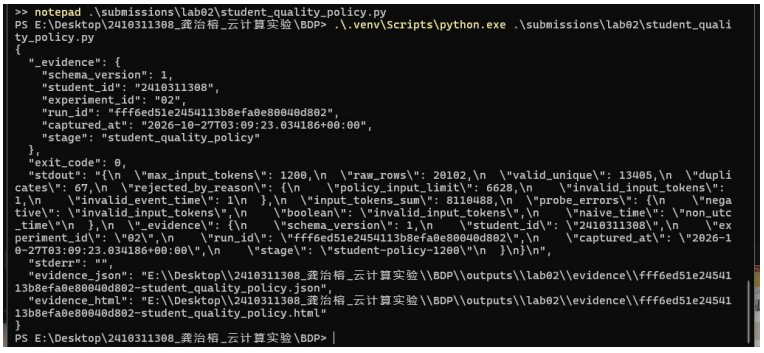
diff 只有一行：MAX_INPUT_TOKENS 由 2000 改为 1200

5.4 第二项独立操作：把布尔值样本改成数字字符串

预测：错误类型可能不变，因为契约使用 type(x) is int 的严格判断，字符串 "100" 与布尔值 True 都不是 int。实测确认两者都返回 invalid_input_tokens。



将 boolean 探针的 True 改为字符串 "100"



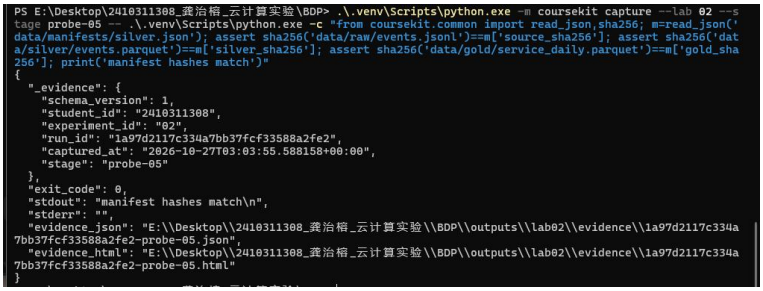
重新运行，boolean 仍返回 invalid_input_tokens



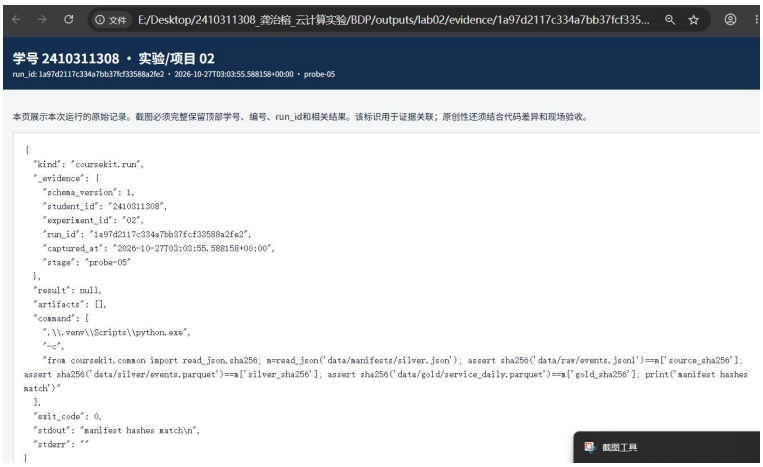
字符串探针证据页，run_id fff6ed51e2454113b8efa0e80040d802

6. 核验文件血缘与最终身份登记

silver.json 记录原始、Silver、Gold 三个摘要。核验时重新计算实际文件哈希并与清单比对。

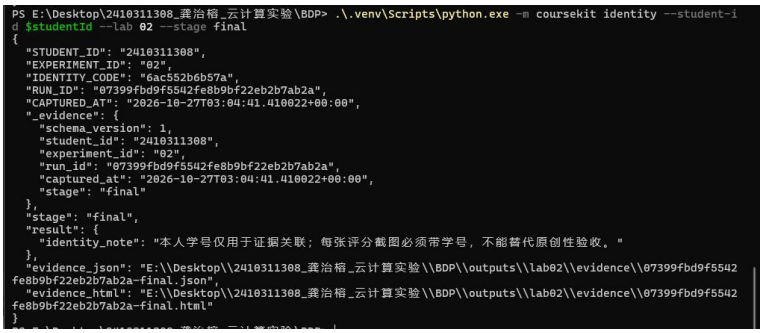


manifest hashes match



probe-05 证据页，run_id 1a97d2117c334a7bb37fcf33588a2fe2

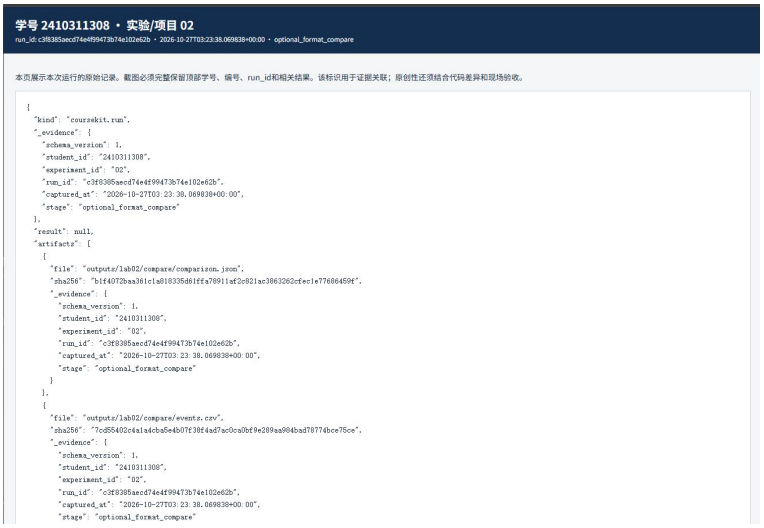
若此处失败，先判断是有意修改了上游文件还是意外覆盖，不要直接改清单里的摘要让校验变绿。若原始文件内容已改变，先重新运行 lab01 以真实对象往返重新核验这份输入，再运行 lab02；若原始文件未变而只有 Silver 或 Gold 损坏，则可直接重跑 lab02。



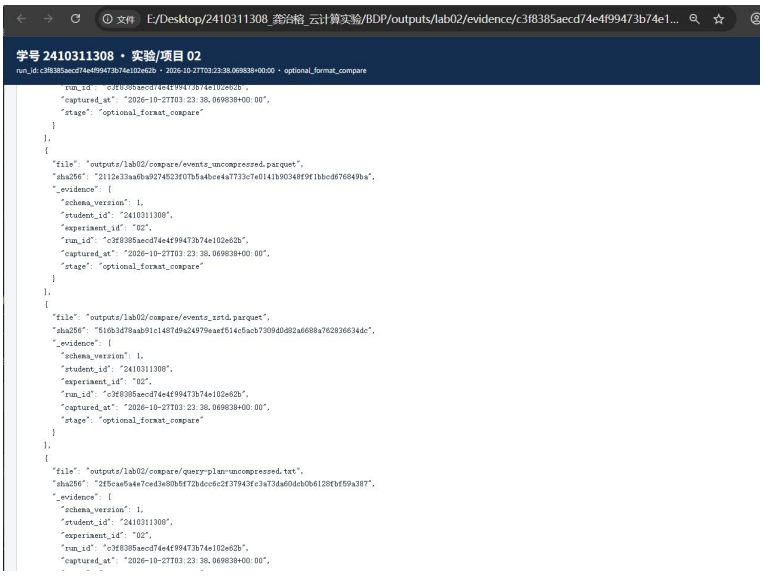
最终身份登记，实验号 02

7. 选做扩展：CSV 与两种 Parquet 的对比

在同一份清洗数据上比较三种存储格式，固定查询与测量范围：同一张 Silver 表、同一条按服务聚合的 SQL、同一进程、同一台机器，每种格式预热 3 次后测量 7 次取中位数，避免首次读盘的冷缓存干扰。三种格式的聚合结果完全一致。



选做运行产物：comparison.json 与 events.csv



未压缩与 Zstd 两个 Parquet 文件及查询计划



三种格式的字节数与耗时，results_identical 为 true

三、实验结果与分析

1. 前置数据与来源原始文件 data/raw/events.jsonl 由实验 01 按 seed=42 生成，共 20102 行，3892184 字节，SHA-256 为 0d93bcd25e28047f804238c220bdb5b1d2dadf328647c07de85ba0104e8e2854，与 raw.json 记录一致。本章未重新生成数据，直接复用该文件。

2. 数据契约

字段：event_id、event_time、service、model、input_tokens、output_tokens、latency_ms、status 共 8 个，缺任一即拒绝为 missing_field。类型：input_tokens、output_tokens、latency_ms 必须是严格整数，使用 type(x) is int 判断。范围：三个数值字段必须大于等于 0。时间：event_time 必须是带时区的 UTC ISO 格式，utcoffset() 必须为 0，无时区或非零偏移分别拒绝。枚举：status 只允许 ok 与 error。重复政策：同 event_id 且内容完全相同计入重复数；同 event_id 但内容不同则报冲突并停止发布，不默默删去。

3. 六行小表的手工推演

按到达顺序逐行处理：第 1 行 a/10 合法，保留；第 2 行 b/20 合法，保留；第 3 行 a/10 与第 1 行完全相同，计入重复；第 4 行 c/-1 因负值被隔离；第 5 行 d/40 因时间为 not-a-time 被隔离；第 6 行 e/30 合法，保留。

手工结果为：合法独特数 3（a、b、e），重复数 1，隔离数 2，保留事件的 input_tokens 总量为 10+20+30 = 60。守恒关系 3+1+2 = 6 成立。

若第 3 行变成 ID=a、Token=99，则它不再是普通重复，而是同 ID 不同内容冲突。本课程规则遇到冲突时停止发布并要求先定义更正政策。业务中可以使用版本、更新时间或纠错事件，但“最后一条一定正确”不是无需论证的默认事实——如果两条记录的采集时间不可靠，简单保留最后一条可能把错误数据留下来、把正确数据丢掉。

4. 基线质量与守恒

原始 20102 行，合法独特 20000 行，重复 100 行，隔离 2 行，守恒 20102 成立。被隔离的两条记录如下：第 20101 行 event_id 为 bad-negative，input_tokens 为 -1，拒绝原因 invalid_input_tokens；第 20102 行 event_id 为 bad-time，event_time 为 not-a-time，拒绝原因 invalid_event_time。两条都在隔离区保留了行号、原因和原始记录，因此“20102 条为什么只剩 20000 条”可以逐条追溯。

5. 存储与 SQL

Silver 共 20000 行 5 个行组,schema 为 8 个字段,其中 event_id、event_time、service、model、status 为 string，三个数值字段为 int64。Gold 每日汇总共 6 行（2 天 × 3 个服务）。Silver 总量为 (20000, 18015640, 6145862)，Gold 按 requests 求和后同为 (20000, 18015640, 6145862)，totals_match 为 true，说明聚合没有丢账也没有重复计数。原始 JSONL 为 3892184 字节，Silver Parquet 为 224693 字节，两者比约为 17 倍。但这个比较不是纯格式性能对照：JSONL 含 100 条重复和 2 条坏行，Silver 已清洗；两者编码方式与压缩方式也不同。因此文件变小不能直接等同于处理变快。

6. 阈值改动与对照

2000 阈值：合法独特 20000，重复 100，隔离 2，输入 Token 总量 18015640，与基线完全一致，说明这条附加规则没有拒绝任何已有合法事件。

1200 阈值：合法独特降至 13405，重复降至 67，输入 Token 总量降至 8110488，另有 6628 条记录因 policy_input_limit 被拒绝。

重复数从 100 降到 67 是关键观察：程序先校验再去重，被新阈值拒绝的原始重复行也进入了拒绝计数，因此不能再按 20000 + 100 + 2 的旧口径核对，必须用 13405 + 67 + 6628 + 1 + 1 = 20102 验证守恒。

误拒绝的代价：被拒绝的 6628 条记录里包含 input_tokens 在 1201 到 2000 之间的合法请求，它们在技术上完全合规，只是超出了本人设定的业务上限。这类记录一旦进入隔离区就不会进入 Silver，下游的用量统计会系统性偏低。技术校验负责判断“这条记录是否可用”，业务政策负责判断“这条记录是否应该计入”，二者必须分开表述，不能把业务上限说成数据非法。

7. 坏样本探针

三个探针都在个人脚本中构造，调用真实 validate_event()，不改动原始文件：负数探针返回 invalid_input_tokens；布尔值探针 True 返回 invalid_input_tokens，因为 bool 虽是 int 的子类但被严格类型判断排除；无时区探针 2026-03-01T12:00:00 返回 non_utc_time。把布尔值换成字符串 "100" 后错误类型不变，仍为 invalid_input_tokens，因为字符串同样不是 int。这说明契约不允许静默转换——如果为了让字符串通过而改用无条件 int()，会掩盖上游接口的类型错误。

8. 文件血缘与恢复

silver.json 记录 source_sha256、silver_sha256、gold_sha256、行数、schema 与 contract_version（本次为 2）。核验脚本重新计算三个实际文件的摘要并与清单比对，输出 manifest hashes match。这个清单帮助后续章节判断文件是否在不知情时发生改变。但它不是数据库事务：它只提供文件内容核验和来源记录，没有实现并发写入原子提交、多文件事务或时间旅行。本课程这一章也没有实现 Iceberg、Delta Lake 或 Hudi 的事务日志与并发提交。

9. 选做扩展：三种存储格式对比

测量结果如下，三种格式的聚合结果完全一致：

CSV：1612574 字节，中位耗时 92.643 毫秒；未压缩 Parquet：1250207 字节，中位耗时 5.028 毫秒；Zstd Parquet：224693 字节，中位耗时 5.329 毫秒。

Parquet 比 CSV 快约 18 倍，但加速主要不来自压缩：未压缩 Parquet 与 Zstd Parquet 的耗时几乎相同。真正的原因是列式存储只需要读取聚合用到的 4 列，且类型信息已写在文件里不需要推断，而 CSV 必须解析全部 8 列的文本并逐列推断类型。Zstd 把文件压到 CSV 的 14%，却没有带来读取加速，反而略慢约 0.3 毫秒，说明压缩节省的是磁盘空间，解压本身有开销。这正好印证手册的提醒：文件变小不等于性能提升。

10. 产物与限制

outputs/lab02 下生成 quality.json、quarantine.json、query-plan.txt、student-policy-2000.json、student-policy-1200.json 与 compare 目录；data/silver/events.parquet、data/gold/service_daily.parquet 与 data/manifests/silver.json 供实验 03、04 继续使用。限制：本实现把 20000 条记录全部放入内存字典去重，数据远超内存时需要外部排序、分布式去重或持久索引；只有单个文件、未按目录分区，不能宣称观察到目录分区裁剪；单次运行的时间测量不能推广为不同硬件或不同数据规模下的性能结论。

11. 运行记录索引

本次实验共 10 次真实运行的证据，均带学号 2410311308 与实验号 02，JSON 与同名 HTML 成对保存，可在证据仓库查看：

https://github.com/yiyun2026/bdp-coursework/lab01/evidence（实验二证据在 lab02 分支目录下）

start cddfee846a1549db9abe3f1190d26d54 身份登记
verify a578991845f842d88e8895d2e2c2db92 主流水线清洗与聚合
probe-01 32a99557827a43d3927919b26ef2a504 Parquet 结构与行组
inspect_gold affccbecbf5c411bb7e60715e35bec69 Silver 与 Gold 总量核对
student_quality_policy 4d92a580e80f43adb23a0303a7eb8303 阈值 2000 基线
student_quality_policy 1d25cc5ea3234e53968ed3cfc5dd931b 阈值 1200
student_quality_policy fff6ed51e2454113b8efa0e80040d802 字符串探针 "100"
probe-05 1a97d2117c334a7bb37fcf33588a2fe2 清单哈希核验
optional_format_compare c3f8385aecd74e4f99473b74e102e62b 选做三种格式对比
final 07399fbd9f5542fe8b9bf22eb2b7ab2a 最终身份登记

四、实验理论回答

为什么 True、"100" 与整数 100 在本契约中的处理可能不同？

契约用的是 type(x) is int 而不是 isinstance(x, int)。Python 中 bool 是 int 的子类，isinstance(True, int) 为真，但 type(True) is int 为假，因此 True 被拒绝。字符串 "100" 的 type 是 str，同样不是 int，也被拒绝。整数 100 的 type 就是 int，通过。这个严格判断是刻意的：如果把 True 当作 1 Token、把 "100" 当作 100 静默转换，上游接口的类型错误就会被掩盖，统计结果出现难以察觉的偏差。

为什么同 ID 内容冲突不能直接当作普通重复计数？

普通重复是同一事件被重复上报，内容完全相同，删掉不影响事实。同 ID 内容不同则意味着两次记录对同一事件给出不同事实，其中至少一条是错的，但仅凭这两行无法判断哪条正确。若直接按重复处理并保留一条，等于在未定义规则的情况下丢掉了可能正确的那条。本课程遇到冲突时停止发布并要求先定义更正政策，正是为了让这个判断显式化，而不是由程序悄悄决定。

原始 JSONL 与清洗 Parquet 的大小比较为什么不是纯格式性能对照？

两者在三个维度上同时变化：内容不同（JSONL 含 100 条重复和 2 条坏行，Parquet 已清洗）、编码不同（文本行 vs 列式二进制）、压缩不同（无压缩 vs Zstd）。文件变小是这三个因素叠加的结果，无法把其中任何一项单独归因于格式。要比较格式本身，必须固定数据内容、只改变存储格式，这正是选做扩展做的事。

列裁剪、谓词下推、行组、目录分区各指什么？本章实际实现了哪些？

列裁剪指只读取查询需要的列；谓词下推指把过滤条件下推到扫描层执行；行组是 Parquet 内部把若干行组织在一起的存储单位，每个行组带列统计；目录分区是按字段值把数据分到

不同目录。本章实现了列裁剪与谓词下推——查询计划中 Projections 只有 input_tokens，Filters 为 service='search'，扫描行数从 20000 降到 3969。本章使用行组（5 个行组，每个 4096 行上限），但没有按目录分区，只写了一个 events.parquet 文件，因此不能宣称观察到目录分区裁剪。

阈值 1200 会丢掉哪些合法业务请求？技术校验与业务政策怎样区分？

会丢掉 6628 条 input_tokens 在 1201 到 2000 之间（以及更高）的记录。这些记录字段完整、类型正确、时间为 UTC，在技术层面完全合规，只是超出了本人设定的业务上限，因此它们进入隔离区而非 Silver，下游统计会系统性偏低。技术校验判断的是“这条记录是否符合数据契约”，结论是客观的、与业务无关；业务政策判断的是“这条记录是否应计入本次统计”，取决于分析目的。二者必须分开表述，不能把业务上限说成数据非法。

文件清单为什么不等于 ACID 事务或完整湖仓表格式？

ACID 要求原子性、一致性、隔离性和持久性，核心是多写者并发下的提交与回滚。本章的清单只记录文件哈希、行数、schema 与契约版本，用于事后核验文件是否被改动，它不控制写入顺序、不支持并发提交、不能回滚、也没有时间旅行。Iceberg、Delta Lake、Hudi 这类表格式提供事务日志与快照管理，本章均未实现。因此清单是数据血缘的最小表达，不能称为事务或表格式。

五、AI 使用披露

本实验在课程允许的范围内使用了 AI 助手，用途限于三类：理解 DuckDB 与 PyArrow 的参数含义、分析报错信息、整理报告的行文结构。AI 没有参与任何一次实际运行，报告中出现的行数、哈希、耗时、run_id 与判定结果均来自本机真实执行。

核对方法有三项。第一，每次截图前打开证据 HTML，确认页面顶部原生显示的学号、实验号 02 与正文 run_id 一致，并把这行身份栏完整截入画面。第二，对 Silver 与 Gold 的总量另用 DuckDB 查询独立核对，确认 requests 之和与 Silver 行数相等、Token 总量一致。第三，把远端课程仓库与本地克隆做版本比对，确认所用手册与评分标准未被改动。

本人独立完成的内容：六行小表的手工推演与四个计数值、同 ID 冲突的处理判断、阈值改为 1200 前后的三项预测、三个坏样本探针的错误类型预测、选做实验中关于加速来源的分析，以及本报告的全部运行取证。

六、实验心得

这次实验让我第一次真切感受到“数据能存下来”和“数据能用”之间的距离。实验 01 证明了字节没变，但这份文件里还混着 100 条重复和 2 条坏行，直接拿去统计就会出偏差。做完实验我才明白，质量契约不是可有可无的文档，而是决定哪些数据能进下游的那道闸门。

六行小表的推演看起来简单，但真正动手算才发现自己在“重复”和“冲突”之间的界线是模糊的。我一开始想的是同 ID 就留一条，后来读手册才意识到：内容完全相同才叫重复，内容不同就是冲突，程序必须停下来而不是自己挑一个。这个区别在真实系统里可能意味着一整批统计要不要发布。

改阈值那一步的结果出乎我的预期。我以为只是合法数据变少，没想到重复数也从 100 掉到了 67。回头想才通：程序先校验再去重，那些被 1200 挡掉的原始重复行根本没机会进入去重环节，于是跑到了拒绝计数里。这让我理解了为什么守恒式必须按实际口径重算，而不是抱着旧的 20000+100+2 不放。更让我在意的是那 6628 条被拒的记录——它们在技术上完全合法，只是被我人为设的上限挡在了外面，下游统计会因此系统性偏低。技术校验和业务政策确实是两件事。

选做部分的对比也让我改变了一个偏见。我原本以为 Parquet 快是因为压缩，结果未压缩

和 Zstd 两种 Parquet 耗时几乎一样，真正的原因是列式存储只读需要的列。压缩把文件压到 CSV 的 14%，速度反而略慢一点。这说明“文件变小”和“查询变快”是两个独立的问题，不能混为一谈。

七、评语

指导教师：刘锦行